

Learn JavaScript: Beginner's Course

Lesson 4: Control Flow

Control Flow - Study Notes

Key Concepts

1. **Conditional Statements** – `if / else` and `switch` let your program choose different paths based on a condition.
2. **Ternary Operator** – A concise, single line form of `if / else`.
3. **Loops** – `for` and `while` repeat a block of code while a condition is true (or for a known number of iterations).
4. **Loop Control** – `break`, `continue`, and loop labels (in some languages) modify the normal flow inside loops.
5. **Scope & Block Statements** – Variables declared inside a conditional or loop block are limited to that block (important for avoiding bugs).

Summary

Control flow is the backbone of any program: it determines **what happens next**.

- **Conditional statements** (`if`, `else if`, `else`, and `switch`) evaluate Boolean expressions and execute the matching code block. `if` is ideal for simple, binary decisions, while `switch` shines when you have many discrete values to compare.
- The **ternary operator** (`condition ? exprIfTrue : exprIfFalse`) provides a compact way to assign a value or execute a short expression based on a condition, keeping the code readable when the logic is simple.

When you need to repeat work, **loops** come into play.

- A **`for` loop** is perfect when you know in advance how many times you need to iterate (e.g., looping through an array).
- A **`while` loop** (or its sibling `do...while`) is used when the number of iterations isn't known beforehand; the loop continues as long as its condition stays true.

Both loops can be interrupted early with `break` or skip to the next iteration with `continue`. Understanding these constructs lets you write programs that react to data, process collections, and perform repetitive tasks efficiently.

Important Points

- **Boolean expressions** must evaluate to `true` or `false`. Anything non-Boolean is usually coerced (e.g., `0`, `null`, `""` ! `false` in JavaScript).
- **`else if` chains** allow multiple mutually exclusive branches; only the first true condition runs.
- **`switch` syntax**:

```
```js
```

```
switch (value) {
 case 1: ...; break;
 case 2: ...; break;
 default: ...;
}
```

- ``break`` prevents “fall through” to the next case.

- Some languages (e.g., Rust, Swift) require exhaustive handling or a ``default`/`_`` case.

- **Ternary operator** is an *expression*, not a statement, so it can be used where a value is required (e.g., inside another expression).

- **Loop headers**:

- ``for` (init; condition; update) { ... }` – ``init`` runs once, ``condition`` is checked before each iteration, ``update`` runs after each iteration.

- ``while` (condition) { ... }` – condition checked before each iteration.

- ``do` { ... } while (condition);`` – body runs **at least once**; condition checked after.

- **``break``** exits the nearest enclosing loop or ``switch``.

- **``continue``** skips the remainder of the current iteration and proceeds to the next loop test.

- **Variable scope**: variables declared with ``let`/`const`` (JS) or inside ``{}`` blocks are limited to that block; ``var`` (JS) is function scoped, which can cause unexpected behavior.

---

## Examples

### 1p ã Conditional Logic + Ternary

```
```javascript
// Determine ticket price based on age
function ticketPrice(age) {
  // if / else version
  if (age < 12) {
    return 5;
  } else if (age >= 65) {
    return 6;
  } else {
    return 10;
  }
  // ternary version (same logic, compact)
  // return age < 12 ? 5 : (age >= 65 ? 6 : 10);
}
```
```

**\*Explanation\***: The function checks three ranges. The ternary version nests two ternary expressions, producing the same result in a single line—useful for simple assignments but less readable when logic grows.

### 2p ã Looping Through an Array (for) and Summing Until a Threshold (while)

```

````javascript
// Sum numbers in an array until the total reaches 100
const numbers = [12, 25, 30, 8, 45, 10];
let total = 0;
// ----- for loop: iterate over every element -----
for (let i = 0; i < numbers.length; i++) {
  console.log(`Index ${i}: ${numbers[i]}`);
}
// ----- while loop: stop when total >= 100 -----
let idx = 0;
while (total < 100 && idx < numbers.length) {
  total += numbers[idx];
  console.log(`Added ${numbers[idx]}, running total = ${total}`);
  idx++;
}
console.log(`Final total: ${total}`);
````

```

**\*Explanation\*:**

- The `for` loop is ideal for visiting each element because the length is known.
- The `while` loop continues adding numbers **\*\*until\*\*** the sum reaches 100 (or the array ends).  
 `break` isn't needed because the loop condition handles termination, but you could `break` early if a special value were encountered.

---

## Quick Review

1. **\*\*When would you prefer a `switch` over multiple `if / else if` statements?\***
2. **\*\*Rewrite the following `if / else` block as a ternary expression:\***  

```

````js
let status;
if (score >= 70) status = 'pass';
else status = 'fail';
````

```
3. **\*\*What is the difference between `while` and `do...while` loops?\***
4. **\*\*How does the `break` statement affect loop execution?\***
5. **\*\*Explain what “fall through” means in a `switch` statement and how to prevent it.\***

---

## Further Reading

- **\*\*Nested loops & complexity\*\*** – handling multi dimensional data structures.
- **\*\*Loop invariants & common pitfalls\*\*** – off by one errors, infinite loops.
- **\*\*Advanced conditional patterns\*\*** – pattern matching (e.g., in Rust, Scala) and guard clauses.
- **\*\*Functional alternatives\*\*** – using `map`, `filter`, `reduce` instead of explicit loops.
- **\*\*Error handling flow\*\*** – `try / catch / finally` as another form of control flow.

